

Bài tập chương VII

Câu 1:

a. Định nghĩa khái niệm tính chịu lỗi trong hệ thống phân tán và liệt kê bốn yêu cầu cơ bản của một hệ thống chịu lỗi.

Tính **chịu lỗi** trong hệ thống phân tán là **khả năng của hệ thống tiếp tục hoạt động một cách chấp nhận được ngay cả khi xảy ra lỗi** ở một hay nhiều thành phần trong hệ thống. Mục tiêu của thiết kế hệ thống chịu lỗi là nhằm **tự phục hồi khi gặp sự cố mà không ảnh hưởng nghiêm trọng đến hiệu năng tổng thể**.

Tính chịu lỗi đặc biệt quan trọng trong hệ thống phân tán vì lỗi có thể xảy ra ở bất kỳ đâu (máy chủ, mạng, tiến trình...), và việc dự phòng, phát hiện, xử lý lỗi giúp giảm thiểu tác động đến dịch vụ đang vận hành.

Bốn yêu cầu cơ bản của một hệ thống chịu lỗi:

1. **Tính sẵn sàng (Availability):**
Hệ thống có thể sẵn sàng phục vụ người dùng **ngay khi có yêu cầu**, tại mọi thời điểm, bất kể có lỗi xảy ra hay không.
2. **Tính đáng tin cậy (Reliability):**
Hệ thống có thể **hoạt động liên tục trong một khoảng thời gian dài mà không xảy ra lỗi**. Tính đáng tin cậy gắn với **độ chính xác và ổn định** của hệ thống.
3. **Tính an toàn (Safety):**
Hệ thống **không gây ra hậu quả nghiêm trọng** ngay cả khi gặp lỗi tạm thời hoặc thực hiện thao tác không chính xác. Đặc biệt quan trọng trong các hệ thống điều khiển hoặc hệ thống quan trọng về mặt an ninh.
4. **Khả năng bảo trì (Maintainability):**
Hệ thống có thể được **sửa chữa một cách dễ dàng, nhanh chóng và tổn thất thấp nhất** khi xảy ra lỗi. Nếu có khả năng **tự động phát hiện và phục hồi**, thì càng nâng cao tính chịu lỗi.

b. Phân biệt tính sẵn sàng (availability) và tính đáng tin cậy (reliability) trong ngữ cảnh tính chịu lỗi, kèm theo ví dụ minh họa cho từng khái niệm.

Phân biệt tính sẵn sàng (Availability) và tính đáng tin cậy (Reliability)

Trong ngữ cảnh **tính chịu lỗi** của hệ thống phân tán, **tính sẵn sàng** và **tính đáng tin cậy** đều là những yếu tố quan trọng, nhưng chúng mang những ý nghĩa khác nhau:

Tiêu chí	Tính sẵn sàng (Availability)	Tính đáng tin cậy (Reliability)
Định nghĩa	Khả năng của hệ thống phục vụ đúng chức năng tại mọi thời điểm người dùng yêu cầu, ngay cả khi có lỗi xảy ra.	Khả năng của hệ thống hoạt động liên tục trong một khoảng thời gian mà không xảy ra lỗi .
Mục tiêu chính	Đảm bảo hệ thống luôn online , sẵn sàng phục vụ người dùng.	Đảm bảo hệ thống chạy ổn định, chính xác , không gặp sự cố trong quá trình hoạt động.
Khi lỗi xảy ra	Hệ thống vẫn duy trì dịch vụ bằng cách chuyển đổi sang phần dự phòng hoặc cơ chế khôi phục .	Hệ thống không được phép lỗi trong toàn bộ quá trình vận hành.
Đo bằng	Tỷ lệ thời gian hệ thống hoạt động đúng chức năng trên tổng thời gian (ví dụ: 99.999%).	Khoảng thời gian liên tục hệ thống chạy mà không bị gián đoạn hoặc hỏng hóc .
Ví dụ minh họa	Một website thương mại điện tử có nhiều máy chủ dự phòng, nếu một máy chủ bị lỗi thì hệ thống vẫn hoạt động bình thường nhờ máy khác. Dù có lỗi, dịch vụ vẫn sẵn sàng .	Một hệ thống điều khiển bay tự động hoạt động suốt 12 giờ liên tục mà không hề phát sinh lỗi hoặc sai sót nào . Đó là đáng tin cậy .

c. Cho ví dụ về một kịch bản lỗi mạng (ví dụ: tràn vùng đệm hoặc mất gói tin). Hãy mô tả cách áp dụng phương pháp dự thừa để đảm bảo hệ thống vẫn có thể phục vụ yêu cầu mà không gián đoạn.

Ví dụ về kịch bản lỗi mạng và cách áp dụng phương pháp dự thừa để đảm bảo tính chịu lỗi

Kịch bản lỗi mạng:

Giả sử trong một hệ thống phân tán, **một gói tin dữ liệu được gửi từ máy chủ A đến máy chủ B**, nhưng **trong quá trình truyền qua mạng, gói tin bị mất do nhiễu tín hiệu hoặc tắc nghẽn mạng**, gây ra **lỗi mất gói tin (packet loss)**.

Lỗi mất gói tin là một lỗi mạng phổ biến, có thể xảy ra nhất thời hoặc chập chờn. Nếu không có biện pháp xử lý, hệ thống có thể không nhận được dữ liệu hoặc phản hồi đúng, dẫn đến **gián đoạn dịch vụ hoặc sai lệch kết quả**.

Cách áp dụng phương pháp dự thừa để đảm bảo hệ thống không gián đoạn

Để đảm bảo hệ thống vẫn phục vụ bình thường trong kịch bản trên, ta có thể **kết hợp nhiều hình thức dự thừa** như sau:

1. Dự thừa thông tin – Ví dụ: Thêm mã kiểm tra lỗi (CRC) hoặc nhân bản dữ liệu

- Trước khi gửi, gói tin được **mã hóa kèm mã phát hiện lỗi** (ví dụ: CRC hoặc Hamming).
- Nếu gói tin đến bị lỗi hoặc không đến, máy nhận có thể:
 - **Phát hiện lỗi nhờ mã CRC.**
 - **Yêu cầu gửi lại gói tin** (nếu mất).
- Ngoài ra, hệ thống có thể lưu **nhiều bản sao dữ liệu** trên các node khác nhau để đảm bảo vẫn có thể truy xuất thông tin khi một node mất kết nối.

Kết quả: Hệ thống có thể **phát hiện và khôi phục dữ liệu lỗi** nhờ thông tin dư thừa.

2. Dư thừa thời gian – Ví dụ: Gửi lại khi không nhận được phản hồi

- Nếu máy chủ B không nhận được gói tin sau một khoảng thời gian, hệ thống sẽ **tự động gửi lại**.
- Kỹ thuật này giúp xử lý lỗi nhất thời như mất gói tin hoặc trễ mạng.

Kết quả: Hệ thống **tự động thử lại**, giúp **duy trì dịch vụ liên tục** ngay cả khi xảy ra lỗi mạng.

3. Dư thừa vật lý – Ví dụ: Có nhiều tuyến đường mạng hoặc máy chủ dự phòng

- Hệ thống có thể thiết kế **nhiều đường truyền mạng vật lý độc lập**.
- Khi một đường truyền gặp sự cố, hệ thống sẽ **tự động chuyển sang tuyến dự phòng**.
- Ngoài ra, có thể cấu hình **các máy chủ dự phòng (redundant servers)** để tiếp quản khi máy chủ chính mất kết nối.

Kết quả: Hệ thống **chuyển hướng sang tài nguyên vật lý khác**, đảm bảo **không gián đoạn dịch vụ**.

d. Phân loại và phân tích ba loại lỗi: nhất thời, chập chờn, và vĩnh cửu.
Trình bày nguyên nhân gốc rễ và tác động của từng loại lỗi đến hoạt động của hệ thống phân tán.

1. Lỗi nhất thời (Transient Fault)

- **Đặc điểm:**
 Lỗi chỉ xảy ra **một lần trong một thời điểm**, sau đó **biến mất**. Nếu thực hiện lại

cùng thao tác thì **lỗi không xuất hiện nữa**.

- **Nguyên nhân gốc rễ:**

- Nhiều sóng tạm thời trong truyền thông.
- Trạng thái không ổn định của hệ thống tại thời điểm xử lý.
- Ví dụ: tràn bộ đệm, nghẽn mạng tạm thời, hoặc lỗi truy cập khi hệ thống đang quá tải nhẹ.

- **Tác động đến hệ thống phân tán:**

- Ít nghiêm trọng nếu hệ thống có cơ chế **thử lại (retry)**.
- Nếu không xử lý đúng, người dùng có thể **gặp lỗi gây hiểu nhầm là mất chức năng**.
- Có thể ảnh hưởng đến **trải nghiệm người dùng và tính sẵn sàng** nếu xảy ra thường xuyên.

- **Ví dụ minh họa:**

Người dùng đăng nhập thất bại ở lần đầu, nhưng lần sau lại thành công mà không thay đổi thông tin → lỗi **nhất thời** trong xử lý xác thực.

2. Lỗi chập chờn (Intermittent Fault)

- **Đặc điểm:**

Lỗi xảy ra **lúc có lúc không**, có thể **theo chu kỳ hoặc ngẫu nhiên**, thường khó xác định nguyên nhân chính xác.

- **Nguyên nhân gốc rễ:**

- Phần cứng yếu (kết nối lỏng, nguồn không ổn định).
- Mạng không ổn định do nhiễu, sóng yếu.
- Lỗi phần mềm do điều kiện ranh giới (race condition, đồng bộ chưa đúng).
- Giao tiếp giữa các node bị gián đoạn nhẹ hoặc lệ thuộc thời gian.

- **Tác động đến hệ thống phân tán:**

- Gây khó khăn trong **kiểm tra và khắc phục** do không lặp lại ổn định.
- Có thể dẫn đến **hành vi không nhất quán**, làm giảm **độ tin cậy** của hệ thống.

- Nếu xảy ra tại các nút quan trọng, có thể khiến hệ thống bị **treo từng phần**, khó phát hiện nguyên nhân.
- **Ví dụ minh họa:**
Một node trong cụm server đôi khi không phản hồi, khiến hệ thống đôi lúc báo lỗi, đôi lúc hoạt động bình thường.

3. Lỗi vĩnh cửu (Permanent Fault)

- **Đặc điểm:**
Lỗi **không tự biến mất, vẫn tồn tại ngay cả khi đã thử khắc phục hoặc khởi động lại**. Phải **thay thế phần bị hỏng hoặc tái cấu trúc** hệ thống mới có thể xử lý.
- **Nguyên nhân gốc rễ:**
 - Hỏng phần cứng vĩnh viễn (ổ cứng lỗi, cáp mạng hỏng...).
 - Lỗi thiết kế phần mềm nghiêm trọng như **deadlock (khóa chết)**.
 - Lỗi logic hệ thống không được kiểm tra đầy đủ (ví dụ: lặp vô hạn, cấu hình sai).
- **Tác động đến hệ thống phân tán:**
 - Có thể **gây sụp đổ hoàn toàn** một phần hoặc toàn bộ hệ thống nếu không có cơ chế dự phòng.
 - Ảnh hưởng nặng đến **tính sẵn sàng, tính đáng tin cậy và tính an toàn** của hệ thống.
 - Gây **mất mát dữ liệu** nếu không có sao lưu hoặc bản sao.
- **Ví dụ minh họa:**
Một ổ cứng lưu trữ dữ liệu bị hỏng vĩnh viễn, hoặc **lỗi khóa chết** khiến giao dịch không bao giờ hoàn tất.

e. Trong một hệ thống phân tán phức tạp (ví dụ: hệ thống thanh toán trực tuyến hoặc điều khiển giao thông thông minh), hãy đánh giá ưu – nhược điểm của các mô hình thất bại: sụp đổ (crash), bỏ sót (omission), thất bại về thời gian (timing) và Byzantine. Trên cơ sở đó, thiết kế một chiến lược chịu lỗi tổng thể (gồm phương pháp phát hiện, ngăn ngừa, và phục hồi lỗi) để tối ưu hóa cả tính sẵn sàng, đáng tin cậy, an toàn và khả năng bảo trì của hệ thống.

I. Đánh giá các mô hình thất bại trong hệ thống phân tán phức tạp

Trong các hệ thống phân tán quy mô lớn và phức tạp như **hệ thống thanh toán trực tuyến** hoặc **điều khiển giao thông thông minh**, các mô hình thất bại sau có ảnh hưởng rất khác nhau:

1. Thất bại sụp đổ (Crash failure)

- **Mô tả:** Thành phần đang hoạt động thì dừng đột ngột và không còn phản hồi.
- **Ưu điểm:** Dễ phát hiện, dễ mô hình hóa.
- **Nhược điểm:** Mất hoàn toàn dịch vụ, ảnh hưởng lớn nếu xảy ra ở node trung tâm (ví dụ: máy chủ cơ sở dữ liệu). Việc khởi động lại có thể gây mất dữ liệu.
- **Tác động:** Ảnh hưởng lớn đến **tính sẵn sàng** và **toàn vẹn dữ liệu**.

2. Thất bại bỏ sót (Omission failure)

- **Mô tả:** Không nhận hoặc không gửi được yêu cầu.
- **Ưu điểm:** Có thể khôi phục nếu triển khai retry logic hoặc xác nhận giao tiếp.
- **Nhược điểm:** Dễ bị nhầm với lỗi mạng tạm thời, khó phân biệt nguyên nhân.
- **Tác động:** Giảm **tính sẵn sàng** và gây ra **trễ** trong hệ thống, có thể ảnh hưởng đến trải nghiệm người dùng (ví dụ trong lớp học trực tuyến hoặc xử lý giao dịch).

3. Thất bại về thời gian (Timing failure)

- **Mô tả:** Hệ thống phản hồi chậm quá mức cho phép.
- **Ưu điểm:** Có thể phát hiện bằng timeout.
- **Nhược điểm:** Dễ bị gây ra bởi tải lớn hoặc tấn công DDoS, làm giảm hiệu suất toàn cục.
- **Tác động:** Ảnh hưởng đến **tính đáp ứng thời gian thực**, đặc biệt nghiêm trọng trong các hệ thống điều khiển giao thông.

4. Thất bại trả về sai kết quả (Incorrect result failure)

- **Mô tả:** Phản hồi sai nhưng vẫn hợp lệ về cú pháp.
- **Ưu điểm:** Ít gặp hơn, có thể kiểm tra bằng cơ chế xác minh kết quả.

- **Nhược điểm:** Rất nguy hiểm nếu không bị phát hiện, ví dụ định tuyến sai, giao dịch tính sai tiền.
- **Tác động:** Ảnh hưởng đến **tính đúng đắn** và **độ tin cậy**, có thể gây **hậu quả nghiêm trọng** nếu không được phát hiện sớm.

5. Thất bại Byzantine (Tùỵ tiện)

- **Mô tả:** Hành vi không thể đoán trước: phản hồi sai, không phản hồi, hoặc phản hồi không hợp lý.
- **Ưu điểm:** Phản ánh thực tế trong môi trường có lỗi phần mềm, tấn công độc hại.
- **Nhược điểm:** **Rất khó phát hiện** và xử lý, cần cơ chế đồng thuận mạnh (ví dụ: thuật toán Byzantine Fault Tolerance - BFT).
- **Tác động:** Gây rối loạn hệ thống, phá hoại dữ liệu hoặc an ninh. Đặc biệt nguy hiểm trong hệ thống tài chính hoặc an toàn mạng.

II. Chiến lược chịu lỗi tổng thể

Để tối ưu **tính sẵn sàng**, **độ tin cậy**, **an toàn**, và **khả năng bảo trì**, chiến lược chịu lỗi cần bao gồm ba thành phần: **phát hiện lỗi**, **ngăn ngừa lỗi**, và **phục hồi lỗi**.

1. Phát hiện lỗi (Fault Detection)

Loại lỗi	Phương pháp phát hiện đề xuất
Sụp đổ	Heartbeat, Watchdog Timer
Bỏ sót	Acknowledgment timeout
Thời gian	Timeout và đo độ trễ trung bình
Sai kết quả	So sánh chéo (redundant calculation), checksum, hash
Byzantine	Voting, đa thực thể (replica voting)

2. Ngăn ngừa lỗi (Fault Prevention)

Phương pháp	Mô tả
Kiểm thử kỹ	Unit test, integration test, chaos testing (Netflix Chaos Monkey)
Kiểm soát tải	Load balancer, autoscaling để tránh overload
Cơ chế xác thực	Giới hạn quyền truy cập và kiểm tra dữ liệu đầu vào

Cách ly tiến trình	Dùng container (Docker), sandbox hóa
Kiểm soát truy cập và mã hóa	Đảm bảo an toàn trước hành vi độc hại (Byzantine)

3. Phục hồi lỗi (Fault Recovery)

Kỹ thuật	Mô tả
Redundancy (dư thừa)	Replication của service, cơ sở dữ liệu đa chủ, hot standby
Checkpointing	Ghi lại trạng thái định kỳ để khôi phục
Rollback / Compensation	Trong giao dịch tài chính: undo hoặc tạo giao dịch đảo chiều
Consensus algorithm	Paxos / Raft / BFT để duy trì nhất quán và ra quyết định an toàn
Self-healing systems	Hệ thống có khả năng tự tái cấu trúc (kết hợp với AI/ML)

III. Mô hình kiến trúc đề xuất

Áp dụng trong hệ thống thanh toán trực tuyến hoặc điều khiển giao thông thông minh:

- **Tầng client:** Thực hiện xác thực đầu vào và timeout kiểm tra phản hồi.
- **Tầng xử lý dịch vụ:**
 - Sử dụng **microservices** kèm container hóa (Docker + Kubernetes).
 - Mỗi dịch vụ quan trọng chạy ở **3 instance** để so sánh kết quả (giảm lỗi Byzantine).
 - Áp dụng **Raft** để đồng thuận trên dữ liệu quan trọng.
- **Tầng dữ liệu:**
 - CSDL nhân bản đa vùng (multi-zone replication).
 - Ghi nhật ký và checkpoint định kỳ.
- **Hạ tầng giám sát:**
 - Sử dụng Prometheus/Grafana để theo dõi và cảnh báo.
 - Các service có tích hợp **circuit breaker** (ví dụ Resilience4J).

Câu 2:

a. Hãy nêu ba loại dư thừa (thông tin, thời gian, vật lý) trong biện pháp che giấu lỗi và cho ví dụ minh họa từng loại.

Ba loại dư thừa trong biện pháp che giấu lỗi

Để đảm bảo tính chịu lỗi trong hệ thống phân tán, người ta sử dụng các biện pháp dư thừa nhằm che giấu lỗi xảy ra, giúp người dùng không nhận thấy hoặc ít nhận thấy sự cố. Có ba loại dư thừa chính:

1. Dư thừa thông tin (Information Redundancy)

- **Khái niệm:** Là việc bổ sung thêm dữ liệu phụ để phát hiện lỗi và phục hồi thông tin bị lỗi.
- **Cách hoạt động:**
 - Chèn thêm bit kiểm tra (parity bit), mã Hamming, CRC (Cyclic Redundancy Check)... vào dữ liệu truyền tải nhằm phát hiện và sửa lỗi trong quá trình truyền.
 - Nhân bản dữ liệu trên nhiều thiết bị lưu trữ để tránh mất mát nếu một bản sao bị hỏng.
- **Ví dụ minh họa:**
 - Trong truyền dữ liệu giữa các nút mạng, bit chẵn lẻ (parity bit) được thêm vào để kiểm tra lỗi đơn giản.
 - Cơ sở dữ liệu replication (sao chép) trên nhiều máy chủ khác nhau để đảm bảo vẫn truy cập được nếu một máy chủ bị lỗi.

2. Dư thừa thời gian (Time Redundancy)

- **Khái niệm:** Là việc thực hiện lặp lại một hành động khi hành động trước đó gặp lỗi hoặc không xác định được kết quả.
- **Cách hoạt động:**
 - Dừng cho các lỗi tạm thời, lỗi giao tiếp hoặc lỗi chập chờn.
 - Hệ thống sẽ tự động thử lại (retry) thao tác nếu không nhận được phản hồi hoặc phản hồi sai.
- **Ví dụ minh họa:**
 - Trong hệ thống giao dịch ngân hàng, khi gửi yêu cầu thanh toán mà không nhận được phản hồi, hệ thống tự động gửi lại yêu cầu sau vài

giây.

- Khi truy vấn đến cơ sở dữ liệu thất bại do nghẽn mạng, ứng dụng có thể tự retry truy vấn sau một khoảng thời gian ngắn.

3. Dự thừa vật lý (Physical Redundancy)

- **Khái niệm:** Là việc bổ sung thêm tài nguyên phần cứng vật lý để thay thế khi có thiết bị chính bị lỗi.
- **Cách hoạt động:**
 - Các thành phần phần cứng (máy chủ, thiết bị mạng, đường truyền...) được cài đặt thêm bản dự phòng (backup).
 - Khi có lỗi xảy ra, hệ thống tự động chuyển sang bản dự phòng mà không làm gián đoạn dịch vụ.
- **Ví dụ minh họa:**
 - Trung tâm dữ liệu sử dụng nhiều máy chủ song song (cluster), nếu một máy chủ gặp sự cố thì một máy khác tự động gánh phần việc.
 - Trong hệ thống điều khiển giao thông, nhiều cảm biến và thiết bị thu phát được lắp tại cùng một vị trí để đảm bảo nếu một cảm biến hỏng vẫn còn cái khác hoạt động.

b. Giải thích khái niệm tiến trình bền bỉ (persistent process) và mô tả cơ chế hoạt động cơ bản khi một tiến trình chính bị lỗi

Tiến trình bền bỉ (Persistent Process) là một giải pháp trong hệ thống phân tán nhằm đảm bảo tính chịu lỗi và tăng độ sẵn sàng của dịch vụ. Mục tiêu chính là duy trì hoạt động liên tục của dịch vụ ngay cả khi một hay nhiều tiến trình trong hệ thống bị lỗi.

1. Khái niệm Tiến trình bền bỉ:

- Là tập hợp các tiến trình được nhân bản và tổ chức thành một nhóm hoạt động như một thể thống nhất.
- Mỗi nhóm có thể gồm nhiều tiến trình chạy trên các máy khác nhau, trong đó **một tiến trình chính đảm nhận xử lý chính thức**, còn các tiến trình còn lại **sẵn sàng thay thế** nếu tiến trình chính bị lỗi.
- Nhóm tiến trình được **nhìn nhận như một thực thể duy nhất** từ bên ngoài. Khi một tiến trình trong nhóm bị lỗi, **nhóm vẫn hoạt động bình thường nhờ các tiến trình còn lại**.

2. Cơ chế hoạt động khi một tiến trình chính bị lỗi:

- Khi **tiến trình chính bị lỗi**, một tiến trình khác trong cùng nhóm sẽ **tự động thay thế** thông qua **giải thuật bầu chọn** hoặc cơ chế điều phối đã thiết kế sẵn.
- Có hai cách tổ chức nhóm:
 - **Kiến trúc ngang hàng (peer-to-peer)**: Tất cả tiến trình có vai trò như nhau. Khi một tiến trình bị lỗi, các tiến trình còn lại bầu chọn để chọn tiến trình thay thế.
 - **Kiến trúc phân cấp (hierarchical)**: Có một tiến trình điều phối (master), nếu tiến trình này lỗi, tiến hành **bầu chọn một điều phối viên mới**.
- Việc phát hiện tiến trình bị lỗi thường được thực hiện bằng cách:
 - **Cơ chế chủ động**: Gửi thông điệp "IS ALIVE" và chờ phản hồi.
 - **Cơ chế thụ động**: Đợi thông điệp từ tiến trình khác trong một thời gian nhất định.
- Nếu **không nhận được phản hồi trong thời gian cho phép**, tiến trình đó được **xác định là bị lỗi**.
- Việc thay thế tiến trình chính diễn ra **trong nội bộ nhóm**, tiến trình bên ngoài (gửi yêu cầu) **không cần biết sự thay đổi**.

c. Cho một hệ thống dịch vụ trực tuyến có lưu trữ dữ liệu quan trọng, hãy đề xuất cách kết hợp dư thừa thông tin (ví dụ CRC, nhân bản dữ liệu) và dư thừa vật lý (ví dụ cụm máy chủ dự phòng) để che giấu lỗi và đảm bảo sẵn sàng phục vụ.

Trong một hệ thống dịch vụ trực tuyến lưu trữ dữ liệu quan trọng (ví dụ như ngân hàng điện tử, thương mại điện tử, y tế số,...), việc đảm bảo tính sẵn sàng cao và che giấu lỗi là cực kỳ quan trọng. Để đạt được mục tiêu này, cần kết hợp **dư thừa thông tin** và **dư thừa vật lý** một cách hợp lý:

1. Dư thừa thông tin

Mục tiêu chính là phát hiện và phục hồi lỗi dữ liệu:

- **Mã phát hiện lỗi và sửa lỗi**:
Sử dụng các kỹ thuật mã hóa như **CRC (Cyclic Redundancy Check)**, mã **Hamming**, hoặc **bit chẵn lẻ** trong truyền tải dữ liệu để **phát hiện lỗi** trong gói tin và thực hiện **truyền lại** hoặc **sửa lỗi tại chỗ** nếu có thể.
- **Nhân bản dữ liệu (Data Replication)**:
Các bản sao dữ liệu nên được lưu trữ ở nhiều nút khác nhau (ví dụ như cụm máy chủ hoặc trung tâm dữ liệu khác nhau) để nếu một bản bị hỏng, hệ thống có thể

chuyển sang dùng bản sao.

Ví dụ:

- Sử dụng mô hình **primary-backup replication**: một bản chính để ghi dữ liệu, nhiều bản sao để đọc hoặc khôi phục khi bản chính lỗi.
- Sử dụng cơ chế **quorum** trong hệ thống phân tán (ví dụ như trong Cassandra hoặc MongoDB): chỉ cần phần lớn các bản sao nhất trí là đủ đảm bảo tính nhất quán và sẵn sàng.

2. Dự thừa vật lý

Mục tiêu là che giấu lỗi phần cứng hoặc lỗi mạng:

- **Cụm máy chủ dự phòng (Failover Clusters):**
Hệ thống nên có các máy chủ vật lý dự phòng (hoặc máy ảo) cho các thành phần chính như: server ứng dụng, server cơ sở dữ liệu, server cân bằng tải.
Ví dụ: nếu một máy chủ xử lý giao dịch bị hỏng, hệ thống sẽ tự động chuyển (failover) sang máy chủ dự phòng tương ứng để tiếp tục phục vụ.
- **Hạ tầng truyền thông dự phòng:**
Cấu hình nhiều đường truyền mạng và nhiều thiết bị mạng (router, switch) chạy song song theo kiểu **chuyển mạch**. Nếu một thiết bị hoặc kênh truyền bị hỏng, luồng dữ liệu có thể chuyển sang thiết bị/kênh khác mà không gián đoạn dịch vụ.
- **Triển khai đa trung tâm dữ liệu (Multi-DC):**
Với các dịch vụ có yêu cầu cao, có thể đặt các máy chủ tại nhiều khu vực địa lý (multi-region), giúp hệ thống vẫn hoạt động bình thường ngay cả khi một khu vực gặp sự cố thiên tai hoặc mất điện diện rộng.

d. So sánh hai kiến trúc nhóm tiến trình (ngang hàng và phân cấp) trong giải pháp tiến trình bền bỉ: phân tích ưu–nhược điểm về mặt độ trễ, phức tạp quản lý và khả năng chịu lỗi khi thành viên nhóm bị sập.

1. Kiến trúc ngang hàng

Trong kiến trúc này, **mọi tiến trình đều bình đẳng**, không có tiến trình trung tâm điều phối.

Ưu điểm:

- **Khả năng chịu lỗi cao:** Nếu một tiến trình trong nhóm bị sập, các tiến trình còn lại vẫn hoạt động và có thể thay thế nhau mà không làm gián đoạn toàn bộ nhóm.
- **Không có điểm lỗi trung tâm:** Việc không có tiến trình chủ giúp loại bỏ rủi ro từ một điểm duy nhất (single point of failure).
- **Tính phân tán cao:** Tăng tính linh hoạt và mở rộng.

Nhược điểm:

- **Độ trễ cao hơn:** Vì mọi quyết định (ví dụ chọn tiến trình xử lý) phải thông qua **giải thuật bầu chọn**, tốn thời gian và lan truyền thông điệp đến toàn nhóm.
- **Quản lý phức tạp hơn:** Việc duy trì trạng thái đồng bộ giữa nhiều tiến trình và thực hiện các thao tác như thêm/rút thành viên phải thông qua **đồng thuận phân tán**, dễ phát sinh lỗi và phức tạp triển khai.
- **Dễ xảy ra lỗi lặp:** Nếu các tiến trình không thống nhất được ai là người xử lý chính, có thể dẫn đến **nhiều tiến trình xử lý cùng yêu cầu**, gây sai lệch.

2. Kiến trúc phân cấp

Trong kiến trúc này, **một tiến trình được chọn làm điều phối viên** (coordinator), các tiến trình khác làm dự phòng và chờ điều phối.

Ưu điểm:

- **Độ trễ thấp hơn:** Khi có yêu cầu, tiến trình điều phối xử lý hoặc giao cho một thành viên cụ thể xử lý. Không cần giải thuật bầu chọn trong tình huống bình thường.
- **Quản lý đơn giản hơn:** Có cấu trúc rõ ràng, vai trò được phân định, phù hợp với hệ thống có quy mô vừa và nhỏ.
- **Phối hợp dễ dàng:** Nhờ có trung tâm điều phối, các thành viên trong nhóm dễ đồng bộ và thống nhất.

Nhược điểm:

- **Điểm lỗi trung tâm:** Nếu tiến trình điều phối bị lỗi, phải kích hoạt **quy trình bầu chọn lại**, tạm thời làm gián đoạn hoạt động.
- **Khả năng chịu lỗi kém hơn ngang hàng:** Khi chưa bầu chọn xong tiến trình điều phối mới, nhóm có thể **không phản hồi được yêu cầu** từ bên ngoài.
- **Khó mở rộng linh hoạt:** Việc thay đổi số lượng thành viên, đặc biệt khi hệ thống lớn lên, sẽ cần tái cấu hình vai trò điều phối.

e. Trong một hệ thống thanh toán phân tán có yêu cầu cực kỳ khắt khe về độ toàn vẹn giao dịch, hãy thiết kế chiến lược chịu lỗi tổng thể kết hợp:

- **Phát hiện lỗi (sử dụng timeout hoặc cơ chế “IS ALIVE”),**
- **Các mức dư thừa (thông tin, thời gian, vật lý),**
- **Giao thức truyền thông tin cậy (điểm-điểm và nhóm),**
- **Cơ chế xử lý thất bại RPC (ít nhất/một lần, nhiều nhất/một lần)...**

Trình bày cách bạn phối hợp các biện pháp trên để tối ưu cả tính sẵn sàng, đáng tin cậy, an toàn và khả năng bảo trì của hệ thống.

1. Phát hiện lỗi (sử dụng timeout hoặc cơ chế “IS ALIVE”)

- **Cơ chế chủ động “IS ALIVE”** được áp dụng giữa các nút trong hệ thống (giữa client-server và giữa các server). Các tiến trình định kỳ gửi heartbeat. Nếu sau **timeout** không nhận được phản hồi → coi là lỗi tạm thời hoặc sập đổ.
- Để **phân biệt lỗi mạng và lỗi nút**, hệ thống sử dụng đa tầng xác minh:
 - Nếu 1 nút không trả lời → hỏi các nút khác xem có kết nối được không → nếu không, đó là lỗi mạng; nếu có, đó là lỗi nút.
- Kết hợp **AI/phân tích thống kê** trên các log trạng thái để phát hiện lỗi lặp lại/tiềm ẩn.

2. Các mức dư thừa (thông tin, thời gian, vật lý)

a. Dư thừa thông tin:

- Sử dụng **mã CRC, mã Hamming**, hoặc **mã Reed-Solomon** để phát hiện và sửa lỗi trong truyền dữ liệu.
- **Nhân bản dữ liệu quan trọng (số dư tài khoản, nhật ký giao dịch)** tại ≥ 3 vị trí (áp dụng quorum 2/3 để quyết định).

b. Dư thừa thời gian:

- **Giao dịch được thực hiện lại khi có lỗi tạm thời**, dựa trên các bản sao lưu trung gian hoặc checkpoint.
- Áp dụng kỹ thuật **retry có kiểm soát (exponential backoff)** để tránh tắc nghẽn.

c. Dư thừa vật lý:

- Tối thiểu **3 server xử lý thanh toán độc lập**, triển khai tại các vị trí địa lý khác nhau.
- Kiến trúc **“Active-Passive + Fencing”**: chỉ 1 server chính xử lý, server phụ luôn đồng bộ trạng thái và sẵn sàng thay thế khi lỗi xảy ra.

3. Giao thức truyền thông tin cậy (điểm-điểm và nhóm)

a. Truyền thông tin cậy điểm-điểm:

- Dựa trên **TCP + xác minh checksum**, kèm số hiệu tuần tự cho mỗi yêu cầu từ client → phát hiện lỗi mất/lặp thông điệp.
- Giao thức ứng dụng bổ sung nhật ký thực hiện (transaction log) tại server để ghi nhớ trạng thái mỗi yêu cầu.

b. Truyền thông nhóm:

- Các server được tổ chức thành **nhóm tiến trình nhân bản chủ động** (active replication), sử dụng giao thức **Atomic Broadcast** hoặc **Paxos/Raft**.
- Áp dụng **quorum voting 2K+1** để chấp nhận quyết định từ số đồng tiến trình không lỗi.

4. Cơ chế xử lý thất bại RPC (ít nhất/một lần, nhiều nhất/một lần)

a. Phía máy chủ:

- **Ghi nhật ký (logging)** trạng thái giao dịch tại các mốc: trước khi xử lý, sau khi xử lý, trước khi gửi kết quả.
- Kết hợp chiến lược:
 - “**Ít nhất một lần**” (**At-least-once**): đảm bảo giao dịch luôn được xử lý dù phải thực hiện lại.
 - “**Nhiều nhất một lần**” (**At-most-once**): kết hợp gán **ID yêu cầu** duy nhất, kiểm tra log trước khi thực hiện → tránh lặp nguy hại (như chuyển tiền 2 lần).

b. Phía máy khách:

- Dùng **timeout + số hiệu tuần tự yêu cầu**.
- Gửi lại yêu cầu nếu không có xác nhận, nhưng phải kèm ID yêu cầu để máy chủ kiểm tra log (tránh xử lý lại).
- Tùy vào loại yêu cầu:
 - **Vô hại** (truy vấn số dư) → có thể xử lý lại.
 - **Nguy hại** (chuyển tiền) → chỉ gửi lại nếu chưa nhận xác nhận.

5. Kết hợp tổng thể để tối ưu:

Tiêu chí

Biện pháp phối hợp cụ thể

Tính sẵn sàng	Nhân bản tiến trình theo kiến trúc phân cấp (leader–follower), dùng thuật toán bầu chọn nhanh để thay thế node hỏng (như Raft).
Đáng tin cậy	Sử dụng giao thức cam kết 2 pha hoặc 3 pha , tích hợp timeout + nhật ký để rollback khi có lỗi giữa chừng.
An toàn	Kiểm tra đồng bộ chữ ký số, mã hóa , và đối chiếu số hiệu yêu cầu + timestamp .
Bảo trì	Quản lý nhóm tiến trình bằng máy chủ thành viên tập trung + giám sát lỗi qua dashboard tập trung. Cho phép scale-out hoặc thay thế node dễ dàng.

Câu 3 :

a. Hãy định nghĩa cam kết nguyên tử (atomic commit) trong giao tác phân tán và nêu tên ba kiểu giao thức cam kết (một pha, hai pha, ba pha).

Cam kết nguyên tử trong giao tác phân tán là một yêu cầu đảm bảo rằng **một giao tác phân tán phải được thực hiện thành công trên tất cả các thành viên tham gia hoặc bị hủy bỏ hoàn toàn trên tất cả các thành viên**. Điều này có nghĩa là nếu một thành viên nào đó trong hệ thống không thể thực hiện được giao tác thì toàn bộ giao tác phải bị hủy để đảm bảo tính nhất quán và nguyên tử của hệ thống. Đặc tính này thường được đảm bảo thông qua việc cài đặt các **giao thức cam kết nguyên tử (Atomic Commit Protocols – ACP)**.

- Ba kiểu giao thức cam kết : một pha, hai pha, ba pha

b. Mô tả ngắn gọn cơ chế hoạt động của giao thức cam kết hai pha (2PC), bao gồm các bước ở pha biểu quyết và pha quyết định, cũng như cách phục hồi khi có thành viên không phản hồi.

Cơ chế hoạt động của giao thức cam kết hai pha (2PC):

Giao thức cam kết hai pha là một cơ chế đảm bảo **tính nguyên tử** trong giao tác phân tán thông qua hai pha chính: **pha biểu quyết** và **pha quyết định**.

1. Pha biểu quyết (Vote Phase):

- **Bước 1:** Tiến trình điều phối gửi thông điệp **VOTE_REQUEST** đến tất cả các tiến trình thành viên.
- **Bước 2:** Mỗi thành viên kiểm tra xem có thể thực hiện giao tác hay không:
 - Nếu **có thể**, gửi **VOTE_COMMIT** cho điều phối.
 - Nếu **không thể**, gửi **VOTE_ABORT**.

2. Pha quyết định (Decision Phase):

- **Bước 1:** Điều phối viên tổng hợp kết quả biểu quyết:
 - Nếu **tất cả** thành viên gửi **VOTE_COMMIT**, điều phối gửi **GLOBAL_COMMIT**.
 - Nếu **ít nhất một** thành viên gửi **VOTE_ABORT**, điều phối gửi **GLOBAL_ABORT**.
- **Bước 2:** Các thành viên thực hiện cam kết hoặc hủy bỏ giao tác tùy theo thông điệp nhận được.

3. Cách phục hồi khi có thành viên không phản hồi:

- Giao thức 2PC không xử lý tốt trường hợp thành viên **không phản hồi**.
- Trong tình huống đó, điều phối viên hoặc các tiến trình khác có thể bị **phong tỏa**, phải **chờ đợi vô thời hạn**.
- Để phục hồi, hệ thống cần sử dụng **nhật ký (log)** để theo dõi trạng thái và hỗ trợ **phục hồi độc lập** nếu tiến trình bị lỗi khởi động lại.

c. Giả sử bạn đang xây dựng một hệ thống đặt vé máy bay phân tán, nơi mỗi bước (đặt chỗ, thanh toán, phát vé) thực thi trên các nút khác nhau. Hãy chọn giao thức cam kết hai pha hoặc ba pha để đảm bảo tính nguyên tử và giải thích lý do, đồng thời mô tả cách xử lý khi tiến trình điều phối bị lỗi.

Trong hệ thống đặt vé máy bay phân tán gồm ba bước: **đặt chỗ**, **thanh toán**, và **phát vé**, mỗi bước được thực hiện trên một nút khác nhau, để đảm bảo **tính nguyên tử**, **giao thức cam kết ba pha (3PC)** là lựa chọn phù hợp hơn giao thức hai pha (2PC).

Lý do chọn giao thức cam kết ba pha (3PC):

- Trong môi trường thực tế như đặt vé máy bay, lỗi hệ thống hoặc mất kết nối có thể xảy ra.
- **2PC dễ bị phong tỏa**: nếu tiến trình điều phối bị lỗi sau khi các thành viên đã biểu quyết, hệ thống có thể rơi vào trạng thái chờ vô thời hạn.
- **3PC khắc phục được điều này** bằng cách thêm một pha trung gian (PRECOMMIT), đảm bảo rằng các thành viên **vẫn có thể đưa ra quyết định** (cam kết hoặc hủy) mà **không cần chờ điều phối viên**, giảm thiểu nguy cơ **treo hệ thống**.

Cơ chế hoạt động của giao thức cam kết ba pha (3PC):

1. Giai đoạn biểu quyết (Vote Phase):

- Điều phối gửi **VOTE_REQUEST** đến tất cả các thành viên (đặt chỗ, thanh toán, phát vé).
- Mỗi thành viên trả lời:
 - **VOTE_COMMIT**: sẵn sàng thực hiện.
 - **VOTE_ABORT**: không thể thực hiện.

2. Giai đoạn chuẩn bị cam kết (Pre-commit Phase):

- Nếu tất cả đều trả lời **VOTE_COMMIT**, điều phối gửi **PRECOMMIT**.
- Các thành viên chuyển sang trạng thái **READY** và ghi log.

3. Giai đoạn cam kết toàn cục (Global Commit Phase):

- Khi điều phối nhận được tất cả phản hồi từ pha **PRECOMMIT**, nó gửi **GLOBAL_COMMIT**.
- Thành viên thực hiện giao tác và cam kết.

Xử lý khi tiến trình điều phối bị lỗi:

- **Nếu lỗi xảy ra sau khi gửi **PRECOMMIT** nhưng chưa gửi **GLOBAL_COMMIT**:**
 - Các thành viên ở trạng thái **READY** sẽ **liên lạc với nhau** để tham khảo trạng thái.
 - Nếu phát hiện thành viên khác ở trạng thái **COMMIT** hoặc tất cả cùng ở trạng thái **PRECOMMIT** → **tự động thực hiện cam kết**.
 - Nếu phát hiện thành viên ở trạng thái **ABORT** → **hủy giao tác**.
- **Nếu lỗi xảy ra trước **PRECOMMIT**** (ví dụ: mất phản hồi từ một thành viên trong pha vote):
 - Điều phối quyết định **GLOBAL_ABORT** → hệ thống hủy bỏ giao tác.

d. So sánh ưu–nhược điểm của giao thức 2PC và 3PC về các khía cạnh sau:

- Khả năng phong tỏa (blocking) khi coordinator hoặc participant sập,
- Hiệu năng (số lượng thông điệp trao đổi),

◦ Độ phức tạp của nhật ký (logging) và phục hồi.

♦ 1. Khả năng phong tỏa (blocking) khi coordinator hoặc participant sập

Tiêu chí	2PC (Hai pha)	3PC (Ba pha)
Coordinator sập	Phong tỏa: Nếu điều phối viên sập sau khi các thành viên đã gửi VOTE_COMMIT, các thành viên phải chờ vô thời hạn để nhận quyết định cuối cùng (GLOBAL_COMMIT/GLOBAL_ABORT).	Không phong tỏa: Các thành viên có thể tự quyết định (hủy/cam kết) dựa trên trạng thái của nhau sau timeout.
Participant sập	Nếu một thành viên sập, coordinator có thể bị treo trong trạng thái chờ phản hồi. Không thể quyết định được giao tác.	Coordinator có thể tiếp tục gửi quyết định cuối cùng cho các thành viên còn lại. Khi thành viên sập phục hồi, có thể dựa vào trạng thái PRECOMMIT để tự hoàn tất.
Tổng thể	Dễ bị phong tỏa trong trường hợp lỗi.	Khả năng không phong tỏa cao hơn nhờ thiết kế trạng thái trung gian (PRECOMMIT) và cơ chế tham khảo.

♦ 2. Hiệu năng (số lượng thông điệp trao đổi)

Tiêu chí	2PC (Hai pha)	3PC (Ba pha)
Số vòng thông điệp	2 vòng: 1. VOTE_REQUEST → VOTE_COMMIT/ABORT 2. GLOBAL_COMMIT/ABORT	3 vòng: 1. VOTE_REQUEST → VOTE_COMMIT/ABORT 2. PRECOMMIT 3. GLOBAL_COMMIT
Tổng số thông điệp	Tối thiểu $2 \times N$ (với N là số thành viên)	Tối thiểu $3 \times N$
Hiệu năng	Hiệu năng tốt hơn do ít thông điệp hơn, độ trễ thấp hơn.	Hiệu năng thấp hơn do số lượng thông điệp và bước đồng bộ nhiều hơn.

♦ 3. Độ phức tạp của nhật ký (logging) và phục hồi

Tiêu chí	2PC (Hai pha)	3PC (Ba pha)
----------	---------------	--------------

Nhật ký ghi trạng thái	Ghi các trạng thái INIT, VOTE_COMMIT, GLOBAL_COMMIT/ABORT.	Ghi thêm trạng thái PRECOMMIT để theo dõi quá trình chuẩn bị cam kết.
Phục hồi sau lỗi	Phức tạp hơn: thành viên khởi động lại không thể biết nên cam kết hay hủy nếu không nhận được quyết định.	Đơn giản hơn: thành viên có thể dựa trên trạng thái PRECOMMIT và trạng thái của các thành viên khác để quyết định.
Tổng thể	Đơn giản hơn về ghi nhật ký nhưng dễ rơi vào tình trạng không thể phục hồi chính xác khi mất thông điệp quyết định.	Phức tạp hơn về ghi chép nhưng khả năng phục hồi chính xác cao hơn , ít phụ thuộc vào coordinator.

e. Thiết kế một giải pháp cam kết phân tán tổng thể cho một hệ thống ngân hàng trực tuyến với yêu cầu cao về độ sẵn sàng và không phong tỏa:

1. Chọn và kết hợp giao thức 2PC/3PC hoặc mở rộng thêm pha “tăng cường” nếu cần.
2. Đề xuất cơ chế logging và phục hồi khi cả coordinator và participants đều có thể lỗi.
3. Trình bày cách cân bằng giữa tính nhất quán (atomicity), độ sẵn sàng (liveness) và hiệu năng (throughput).

Hãy mô tả chi tiết các bước phối hợp giữa các nút, các trạng thái chuyển đổi, và cách hệ thống đưa ra quyết định cuối cùng trong mọi kịch bản lỗi.

1. Lựa chọn và kết hợp giao thức: Mở rộng giao thức 3PC với pha “tăng cường phục hồi”

Mục tiêu:

- Đảm bảo **không phong tỏa** (non-blocking).
- **Cam kết nguyên tử** (atomic commit).
- Duy trì **độ sẵn sàng cao** ngay cả khi **Coordinator hoặc Participants bị lỗi**.

Giải pháp: Sử dụng 3PC (Three-Phase Commit) làm nền tảng, mở rộng thêm:

- **Pha tăng cường (Enhanced Recovery Phase)** để hỗ trợ phục hồi khi coordinator bị lỗi trong PRECOMMIT.
- Kết hợp thêm cơ chế **quorum-based voting** nếu hệ thống lớn, nhằm tăng khả năng quyết định ngay cả khi thiếu một vài thành viên.

2. Cơ chế logging và phục hồi

Logging:

- Mỗi tiến trình (coordinator hoặc participant) ghi **write-ahead log (WAL)** vào bộ nhớ bền (ví dụ: ổ đĩa SSD, log service).
- Log ghi **từng trạng thái quan trọng**:
 - INIT, VOTE_REQUEST, VOTE_COMMIT, VOTE_ABORT
 - PRECOMMIT, GLOBAL_COMMIT, GLOBAL_ABORT

Phục hồi:

Nếu coordinator bị lỗi:

- Khi phục hồi, coordinator đọc trạng thái từ log:
 - Nếu log đã ghi PRECOMMIT và chưa có GLOBAL_COMMIT: tiếp tục gửi GLOBAL_COMMIT.
 - Nếu log chỉ có VOTE_REQUEST: hủy giao tác (gửi GLOBAL_ABORT).
 - Nếu log rỗng: abort mặc định.

Nếu participant bị lỗi:

- Khi khởi động lại:
 - Nếu ở trạng thái READY, nhưng chưa có PRECOMMIT: liên hệ các participant khác.
 - Nếu ai đó trả lời ABORT → tự abort.
 - Nếu trả lời PRECOMMIT hoặc COMMIT → tự điều chỉnh theo đó.

- Nếu ở trạng thái **PRECOMMIT**, mà không nhận được **GLOBAL_COMMIT**: hỏi trạng thái các thành viên khác.
 - Nếu ai đó COMMIT → tự COMMIT.
 - Nếu tất cả là PRECOMMIT → tự COMMIT.

3. Cân bằng giữa Nhất quán – Sẵn sàng – Hiệu năng

Yếu tố	Hành động thiết kế
Tính nhất quán (Atomicity)	Sử dụng giao thức 3PC đảm bảo mọi nút đều thống nhất thực hiện hoặc hủy giao tác.
Độ sẵn sàng (Liveness)	Cơ chế không phong tỏa (non-blocking), phục hồi độc lập từng nút, trao đổi trạng thái giữa các thành viên giúp tiến tới quyết định cuối cùng mà không cần coordinator luôn sẵn sàng.
Hiệu năng (Throughput)	Tối ưu số vòng lặp bằng cách: batch các VOTE_REQUEST , sử dụng timeout hợp lý, chỉ chọn 3PC khi thực sự cần, còn lại có thể fallback sang 2PC nếu ít nút.

4. Chi tiết phối hợp giữa các nút:

Thành phần:

- **Coordinator** (Điều phối)
- **Participants** (Thành viên)

Các bước và chuyển trạng thái:

Giai đoạn	Coordinator	Participant
INIT	Nhận yêu cầu từ client	-
Pha 1: VOTE	Gửi VOTE_REQUEST	Nhận, ghi log READY nếu đồng ý, hoặc ABORT nếu không thể
Pha 2: PRECOMMIT	Nếu tất cả trả lời VOTE_COMMIT , gửi PRECOMMIT	Nhận và ghi log PRECOMMIT , phản hồi xác nhận
Pha 3: COMMIT	Nhận phản hồi từ tất cả, gửi GLOBAL_COMMIT	Nhận, ghi log và thực hiện COMMIT

Failure Handling	Nếu không đủ phản hồi → gửi GLOBAL_ABORT	Tự hỏi các participant khác nếu timeout để xác định hành động tiếp theo
-------------------------	---	---

5. Kịch bản lỗi và quyết định cuối cùng

Tình huống	Hành động
Coordinator bị lỗi trước khi gửi PRECOMMIT	Participants timeout và tự hủy bỏ giao tác
Coordinator bị lỗi sau khi gửi PRECOMMIT	Participants giao tiếp với nhau , nếu đồng nhất → cam kết
Participant lỗi sau READY , chưa nhận PRECOMMIT	Hỏi các thành viên khác → đồng bộ trạng thái
Participant lỗi sau PRECOMMIT	Hỏi các thành viên khác, nếu tất cả là PRECOMMIT → tự cam kết

Câu 4:

a. Hãy nêu định nghĩa phục hồi lùi và phục hồi tiến, đồng thời liệt kê ưu – nhược điểm cơ bản của mỗi phương pháp.

Định nghĩa:

- **Phục hồi lùi (Backward Recovery):**
Là kỹ thuật đưa hệ thống từ trạng thái lỗi trở về trạng thái không lỗi trước đó, thông qua việc phục hồi lại từ các điểm kiểm tra (checkpoint) đã lưu trước khi xảy ra lỗi.
- **Phục hồi tiến (Forward Recovery):**
Là kỹ thuật đưa hệ thống từ trạng thái lỗi sang một trạng thái mới không lỗi để tiếp tục hoạt động, bằng cách phát hiện và sửa lỗi mà không quay về trạng thái cũ.

So sánh cơ bản

Tiêu chí	Phục hồi lùi	Phục hồi tiến
----------	--------------	---------------

Cách hoạt động	Quay lại trạng thái trước khi lỗi	Sửa lỗi để chuyển sang trạng thái mới
Yêu cầu dự đoán lỗi	Không	Có
Hiệu năng	Tốn hiệu năng do ghi trạng thái và phục hồi	Hiệu năng tốt nếu lỗi sửa được nhanh
Khả năng ứng dụng	Rộng rãi, tổng quát	Hạn chế, chuyên biệt
Điểm mạnh	Tính tổng quát, dễ tích hợp	Không quay lui, phản ứng nhanh nếu được hỗ trợ
Điểm yếu	Phục hồi dây chuyền, không xử lý được hành vi không đảo ngược	Khó thiết kế, cần logic sửa lỗi

b. Giải thích vai trò của điểm kiểm tra (checkpoint) và ghi nhật ký thông điệp (message logging) trong cơ chế phục hồi lùi, và tại sao kết hợp cả hai lại giúp giảm chi phí lưu trạng thái.

1. Vai trò của điểm kiểm tra (Checkpoint)

Điểm kiểm tra là kỹ thuật ghi lại trạng thái cục bộ của mỗi tiến trình tại một thời điểm xác định, giúp hệ thống có thể phục hồi lùi về trạng thái trước khi xảy ra lỗi.

- Trong hệ thống phân tán, nếu các điểm kiểm tra của các tiến trình được phối hợp (gọi là bản sao phân tán nhất quán), thì việc phục hồi lùi sẽ đơn giản và tránh hiện tượng phục hồi dây chuyền.
- Nếu chỉ dùng điểm kiểm tra độc lập, thì có thể dẫn đến việc phải quay lui nhiều lần để tìm trạng thái toàn cục hợp lệ → gây tốn kém tài nguyên và thậm chí phải chạy lại từ đầu.

2. Vai trò của ghi nhật ký thông điệp (Message Logging)

Ghi nhật ký thông điệp là kỹ thuật ghi lại tất cả các thông điệp đã gửi và nhận để có thể phát lại các thông điệp sau khi khôi phục từ một điểm kiểm tra.

- Giúp tiến trình phục hồi lại trạng thái hiện tại bằng cách khởi động từ điểm kiểm tra và phát lại thông điệp đã ghi thay vì cần một điểm kiểm tra mới hơn.
- Nhờ đó, hệ thống vẫn có thể đạt trạng thái nhất quán mà không cần ghi trạng thái quá thường xuyên.

3. Tại sao kết hợp cả hai lại giúp giảm chi phí lưu trạng thái?

- Chỉ sử dụng điểm kiểm tra đòi hỏi phải ghi trạng thái thường xuyên để giảm mất mát → tốn tài nguyên (bộ nhớ ổn định, thời gian ghi).
- Chỉ dùng ghi nhật ký thì nếu không cẩn thận, có thể xảy ra hiện tượng tiến trình mờ côi và trạng thái không nhất quán.

Kết hợp điểm kiểm tra và ghi nhật ký giúp:

- Giảm tần suất cần checkpoint vì có thể phục hồi từ checkpoint cũ và phát lại thông điệp.
- Tăng hiệu quả phục hồi, vì tiến trình không phải quay lui quá xa (do có thể phát lại thông điệp để đi tiếp).
- Tránh hiện tượng phục hồi dây chuyền hoặc tiến trình mờ côi bằng cách lưu các phụ thuộc hợp lý.

c. Giả sử bạn đang xây dựng một hệ thống đặt vé máy bay phân tán, nơi mỗi bước (đặt chỗ, thanh toán, phát vé) thực thi trên các nút khác nhau. Hãy chọn giao thức cam kết hai pha hoặc ba pha để đảm bảo tính nguyên tử và giải thích lý do, đồng thời mô tả cách xử lý khi tiến trình điều phối bị lỗi.

1. Đặc điểm của hệ thống và yêu cầu

- Hệ thống phân tán giao dịch ngân hàng: dữ liệu quan trọng, cần độ tin cậy cao và không được mất mát dữ liệu.
- Yêu cầu phục hồi nhanh nhưng không làm giảm hiệu năng hệ thống do việc lưu trữ trạng thái quá thường xuyên.

2. Đề xuất chính sách lập điểm kiểm tra

♦ Loại điểm kiểm tra:

Điểm kiểm tra phối hợp (coordinated checkpointing)

→ Vì:

- Đảm bảo trạng thái toàn cục nhất quán, tránh hiện tượng phục hồi dây chuyền.
- Đơn giản hóa việc phục hồi.
- Phù hợp với hệ thống có giao dịch quan trọng như ngân hàng.

♦ Tần suất điểm kiểm tra:

- Thực hiện định kỳ theo chu kỳ (VD: mỗi 5–10 phút) hoặc khi có một lượng giao dịch nhất định (VD: sau mỗi 1000 giao dịch).
- Có thể điều chỉnh linh hoạt (adaptive checkpointing): nếu hệ thống ít thay đổi, giảm tần suất để tiết kiệm tài nguyên.

♦ Cơ chế thực hiện:

- Áp dụng giải thuật ảnh chụp phân tán (distributed snapshot) hoặc giao thức phong tỏa hai pha (two-phase commit) để đồng bộ các tiến trình.
- Ảnh lũy tiến (incremental propagation): chỉ những tiến trình phụ thuộc mới tham gia checkpoint → giảm overhead.

3. Đề xuất chiến lược ghi nhật ký

♦ Chiến lược ghi nhật ký:

Ghi nhật ký thông điệp bi quan (pessimistic message logging)

→ Vì:

- Đảm bảo mỗi thông điệp quan trọng được ghi vào bộ nhớ ổn định ngay khi được nhận → thông điệp không bị mất khi xảy ra lỗi.
- Tránh tiến trình mờ côi và đơn giản hóa phục hồi.

♦ Cách ghi nhật ký:

- Mỗi tiến trình khi gửi hoặc nhận thông điệp đều ghi lại vào bộ nhớ ổn định (ổ đĩa/log server): gồm nội dung, số thứ tự, thời gian, và quan hệ nhân quả.
- Sau khi checkpoint, có thể xóa các thông điệp đã ổn định để tiết kiệm dung lượng lưu trữ.

4. Cách đảm bảo phục hồi nhanh chóng và tối ưu hiệu năng

Mục tiêu	Giải pháp
Phục hồi nhanh	Từ điểm kiểm tra phối hợp gần nhất → không cần dò phụ thuộc, không quay lui nhiều tầng
Trạng thái nhất quán	Nhờ checkpoint phối hợp và ghi nhật ký bi quan → luôn đảm bảo không mờ côi tiến trình
Tối ưu hiệu năng	- Giảm số lần checkpoint bằng logging - Ảnh lũy tiến giảm số tiến trình tham gia checkpoint - Chỉ ghi thông điệp quan trọng vào log ổn định

d. So sánh hai kỹ thuật điểm kiểm tra độc lập và điểm kiểm tra phối hợp về:

- Khả năng xác định “đường phục hồi” (recovery line),
- Tác động đến hiệu năng trong thời gian hoạt động bình thường,
- Độ phức tạp khi triển khai.

♦ 1. Khả năng xác định “đường phục hồi” (recovery line)

Tiêu chí	Điểm kiểm tra độc lập	Điểm kiểm tra phối hợp
Đặc điểm	Mỗi tiến trình lưu trạng thái cục bộ riêng biệt, không đồng bộ với tiến trình khác.	Tất cả tiến trình đồng bộ lưu trạng thái cùng lúc, tạo ra trạng thái toàn cục nhất quán.
Khả năng xác định đường phục hồi	Khó xác định. Có thể xảy ra quay lui dây chuyền, phải lùi về rất xa để đạt được trạng thái nhất quán toàn cục.	Dễ xác định vì trạng thái đã đảm bảo nhất quán toàn cục. Không xảy ra hiệu ứng quay lui dây chuyền.

Ví dụ điển hình	Nếu tiến trình P2 phục hồi về trạng thái mà nhật ký nhận thông điệp m2, nhưng P1 không ghi nhận gửi m2, phải quay lui tiếp.	Trạng thái của P1 và P2 được chụp cùng lúc, đảm bảo m2 được ghi nhận nhất quán.
------------------------	---	---

Kết luận: Kỹ thuật phối hợp tốt hơn rõ rệt về khả năng xác định “đường phục hồi”.

♦ 2. Tác động đến hiệu năng trong thời gian hoạt động bình thường

Tiêu chí	Điểm kiểm tra độc lập	Điểm kiểm tra phối hợp
Hiệu năng bình thường	Tốt hơn vì không cần đồng bộ, các tiến trình tự do chọn thời điểm điểm kiểm tra.	Chậm hơn do phải đồng bộ tất cả tiến trình tại thời điểm điểm kiểm tra.
Tác động	Tác động nhỏ đến hoạt động bình thường, phù hợp cho hệ thống nhạy cảm với độ trễ.	Làm giảm hiệu năng tạm thời do đồng bộ và phong tỏa (dù có thể dùng giải pháp không phong tỏa).

Kết luận: Kỹ thuật độc lập có lợi hơn về hiệu năng trong thời gian hoạt động bình thường.

♦ 3. Độ phức tạp khi triển khai

Tiêu chí	Điểm kiểm tra độc lập	Điểm kiểm tra phối hợp
Đặc điểm triển khai	Cần ghi nhận phụ thuộc giữa các tiến trình (ai gửi gì cho ai, khi nào) → phức tạp trong xử lý quay lui.	Dễ triển khai hơn vì chỉ cần đồng bộ theo giao thức đơn giản (như hai pha, hoặc snapshot không phong tỏa).
Tính phức tạp	Cao, vì phải xử lý phụ thuộc nhân quả, chuỗi quay lui, và tìm trạng thái nhất quán thủ công.	Thấp hơn, nhờ sự hỗ trợ của giải thuật đồng bộ toàn cục và phục hồi dễ hơn.

Kết luận: Kỹ thuật phối hợp đơn giản hơn trong triển khai thực tế, nhất là trong hệ thống lớn.

e. Thiết kế một cơ chế phục hồi tổng thể cho một hệ thống IoT phân tán (ví dụ: mạng cảm biến môi trường) bao gồm:

1. Phương án sao lưu trạng thái (checkpoint) và ghi nhật ký thông điệp,

2. Cơ chế phát hiện lỗi và kích hoạt phục hồi,
3. Quy trình khôi phục khi một nút hoặc vùng mạng bị mất,
4. Cách tối ưu cân bằng giữa tính sẵn sàng, tính nhất quán và chi phí bằng thông/độ trễ.

Mô tả chi tiết các bước, điều kiện khởi tạo điểm kiểm tra, cách tái gửi thông điệp mất, và cách lựa chọn “đường phục hồi” phù hợp.

1. Phương án sao lưu trạng thái (checkpoint) và ghi nhật ký thông điệp

Checkpoint phối hợp (Coordinated Checkpointing)

- Tất cả các cảm biến định kỳ lưu trạng thái cục bộ vào bộ nhớ ổn định (ổ đĩa, SSD, hoặc thiết bị lưu trữ từ xa).
- Sử dụng giải thuật hai pha:
 - Giai đoạn 1: tiến trình điều phối gửi **CHECKPOINT_REQUEST** tới các node phụ thuộc;
 - Giai đoạn 2: sau khi tất cả phản hồi, gửi **CHECKPOINT_DONE** để tiếp tục xử lý.
- Đảm bảo tạo ra một trạng thái toàn cục nhất quán, tránh hiệu ứng domino quay lui.

Ghi nhật ký thông điệp (Message Logging)

- Tiến trình gửi ghi lại thông điệp trước khi gửi (sender-based logging).
- Tiến trình nhận ghi lại thông điệp trước khi xử lý (receiver-based logging).
- Sử dụng giao thức ghi nhật ký bi quan (Pessimistic Logging):
 - Thông điệp chỉ được xử lý sau khi đã ghi vào bộ nhớ ổn định, giúp tránh tiến trình mò côi.

2. Cơ chế phát hiện lỗi và kích hoạt phục hồi

Cơ chế phát hiện lỗi

- Cảm biến lân cận thực hiện kiểm tra **IS ALIVE** định kỳ thông qua heartbeat (nhịp báo sống).

- Nếu không nhận được phản hồi trong một khoảng thời gian **timeout**, xem nút là lỗi.

Kích hoạt phục hồi

- **Nút điều phối vùng hoặc nút gateway sẽ kích hoạt quy trình phục hồi khi:**
 - Mất liên lạc với cảm biến (lỗi phần mềm/phần cứng);
 - Phát hiện trạng thái lỗi từ báo cáo nhật ký hệ thống.

3. Quy trình khôi phục khi một nút hoặc vùng mạng bị mất

Khôi phục một nút đơn

- Tiến trình khởi động lại từ bản sao checkpoint gần nhất (đường phục hồi).
- Dựa trên các thông điệp đã ghi trong nhật ký để tái gửi lại những thông điệp đã thất lạc.
- Nếu nút bị lỗi vật lý, có thể kích hoạt node dự phòng (redundant node) đã được cấu hình sẵn.

Khôi phục vùng mạng

- Tái cấu trúc mạng con: chọn một gateway thay thế từ nút gần kề.
- Đồng bộ trạng thái từ các checkpoint phân tán đã lưu từ các node khác.
- Sử dụng cơ chế ảnh lũy tiến (transitive dependency) để xác định các node liên quan cần phục hồi đồng thời.

4. Cân bằng giữa tính sẵn sàng, nhất quán và chi phí băng thông/độ trễ

Mục tiêu	Giải pháp cân bằng
Tính sẵn sàng	- Sử dụng checkpoint phối hợp giúp hệ thống phục hồi nhanh. - Có node dự phòng tự động nhận nhiệm vụ nếu node chính bị lỗi.

Tính nhất quán	- Ghi nhật ký thông điệp và checkpoint đồng bộ. - Đảm bảo không xảy ra tiến trình mờ côi bằng logging bị quan.
Chi phí bằng thông/độ trễ	- Giảm khoảng thời gian checkpoint, chỉ checkpoint khi trạng thái có thay đổi lớn. - Tái phát thông điệp từ nhật ký, tránh checkpoint quá thường xuyên.

Chi tiết các bước

A. Điều kiện khởi tạo checkpoint

- Định kỳ (ví dụ mỗi 10 phút) hoặc khi có thay đổi trạng thái quan trọng.
- Khi hệ thống chuyển sang trạng thái có nguy cơ lỗi (ví dụ, pin yếu, độ nhiễu cao).
- Khi có yêu cầu từ tiến trình điều phối.

B. Cách tái gửi thông điệp bị mất

- Dựa trên bản ghi message log, tiến trình gửi tái gửi lại các thông điệp đã gửi từ checkpoint cuối.
- Mỗi thông điệp gắn mã định danh, `sequence_id`, để tránh gửi lặp.
- Nếu thông điệp không được ghi nhật ký $\Rightarrow$ báo lỗi không thể phục hồi (trừ khi dùng mã sửa lỗi).

C. Cách lựa chọn “đường phục hồi” phù hợp

- Lựa chọn checkpoint gần nhất có tính nhất quán toàn cục (tập hợp trạng thái có thể khôi phục không gây xung đột).
- Ưu tiên checkpoint được phối hợp toàn cục hơn checkpoint độc lập (tránh quay lui dây chuyền).
- Có thể sử dụng các thông tin phụ thuộc $INT_i(m) \rightarrow INT_j(n)$ để tính toán đường phục hồi.

